

Titre Professionnel
Concepteur Développeur d'Applications
Niveau 6 (Bac +3)

Dossier de Projet Professionnel
Présenté dans le cadre de la certification RNCP

SOCIAL NETWORK

Développement d'une plateforme sociale full-stack avec messagerie temps réel,
groupes, événements et notifications

Next.js 15	React 19	TypeScript	Prisma ORM
PostgreSQL	Redis/Upstash	Docker	Vercel

Nom du candidat : Christophe Lecart
Centre de formation : Zone01 Rouen Normandie
Session d'examen : Juin 2026
Référentiel RNCP : 37873

Durée du projet : 26/05/2025 – 18/09/2025

SOMMAIRE

1. Présentation du candidat
 - 1.1 Parcours et reconversion
 - 1.2 Compétences mobilisées
 - 1.3 Vision professionnelle
2. Contexte du projet
 - 2.1 Genèse et motivation
 - 2.2 Périmètre fonctionnel
 - 2.3 Enjeux techniques et pédagogiques
3. Objectifs du projet
 - 3.1 Objectifs techniques
 - 3.2 Objectifs CDA / RNCP
4. Spécifications fonctionnelles
 - 4.1 Analyse du besoin
 - 4.2 User Stories (US001 – US038)
 - 4.3 Fonctionnalités principales
 - 4.4 Rôles et permissions
5. Spécifications techniques
 - 5.1 Architecture logicielle
 - 5.2 Organisation des fichiers (Next.js App Router)
 - 5.3 Technologies utilisées
 - 5.4 Modèle de données — 18 entités Prisma
6. Réalisations techniques
 - 6.1 Authentification JWT + OAuth Google
 - 6.2 API REST — 60+ endpoints
 - 6.3 Messagerie temps réel SSE + Redis
 - 6.4 Système de posts, réactions, stories
 - 6.5 Groupes, événements, notifications
 - 6.6 Performances et cache Redis
7. Tests et validation
 - 7.1 Stratégie de test
 - 7.2 Jeux d'essai
 - 7.3 Résultats et couverture
8. Sécurité et RGPD
 - 8.1 Mesures de sécurité
 - 8.2 Conformité RGPD
9. Déploiement
 - 9.1 Infrastructure Docker
 - 9.2 CI/CD et mise en production
10. Recherche technique personnelle
11. Bilan RNCP et conclusion

1. Présentation du candidat

1.1 Parcours et reconversion

1.1 Parcours et reconversion

Je m'appelle Christophe Lecart, j'ai 44 ans et je suis actuellement en formation à Zone01 Rouen Normandie dans le cadre de l'obtention du Titre Professionnel Concepteur Développeur d'Applications (niveau 6 - Bac+3, RNCP 37873).

Mon parcours est marqué par une reconversion professionnelle après plus de vingt ans passés dans l'industrie, notamment au sein de groupes comme SOCOTEC (inspection technique, gestion de la qualité) et KONE (systèmes d'ascenseurs, maintenance industrielle). Ces expériences m'ont permis d'acquérir une rigueur opérationnelle, un sens de la responsabilité, une capacité à travailler sur des systèmes techniques complexes et le respect de procédures strictes.

J'ai intégré Zone01 Rouen Normandie en 2024, une formation fondée sur l'autonomie, le travail en peer-to-peer et l'apprentissage par la pratique. Ce modèle pédagogique me permet de développer une approche structurée du code : organisation de la logique applicative, écriture de code maintenable et travail sur des projets de bout en bout.

Parcours professionnel

Période	Poste / Structure	Compétences transférées
2002 – 2003	LUBRIZOL — Technicien bureau d'études (alternance)	Support technique et participation à des activités d'études
2003 – 2010	KONE __ Technicien hautement qualifié	Travail en environnement technique, respect des procédures, interventions sur systèmes industriels
2010 – 2024	Technicien / Expert — SOCOTEC	Gestion de missions techniques, conformité, analyse réglementaire, travail en équipe avec ingénieurs
2024 – aujourd'hui	Formation Zone01 Rouen Normandie	Full-stack TypeScript, architecture logicielle, sécurité, DevOps

1.2 Compétences mobilisées

Au cours du projet Social Network, j'ai mobilisé un ensemble de compétences couvrant l'intégralité du cycle de développement d'une application web moderne, depuis la conception de l'architecture jusqu'au déploiement en production.

Domaine	Technologies / Pratiques
Frontend	Next.js 15 App Router, React 19, TypeScript, Tailwind CSS
Backend / API	Next.js API Routes, REST, validation Zod, 60+ endpoints
Base de données	Prisma ORM, PostgreSQL (Neon), 18 modèles, 40+ relations
Temps réel	Server-Sent Events (SSE), Redis Upstash, polling fallback
Sécurité	JWT (jose), bcrypt, middleware auth, headers sécurisés
Cloud / Media	Cloudinary (upload images), Neon DB serverless PostgreSQL
DevOps	Docker, Docker Compose, CI/CD GitHub Actions, Vercel
Qualité	Tests intégration, jeux d'essai, Playwright E2E

1.3 Vision professionnelle

Ce projet marque pour moi un tournant : je ne suis plus simplement quelqu'un qui code, mais un développeur capable de concevoir, architecturer, sécuriser et livrer une application complexe de A à Z. Mon profil est à la fois technique et pragmatique — influencé par des années passées à faire fonctionner des systèmes réels dans des environnements contraints.

Je me positionne comme développeur full-stack TypeScript, avec une appétence particulière pour l'architecture back-end, la performance des requêtes, la sécurité des API et les systèmes temps réel. Je suis actuellement en recherche d'alternance de niveau Bac+5 pour consolider et approfondir cette trajectoire.

2. Contexte du projet

2.1 Genèse et motivation

Le projet Social Network est né d'une volonté de construire une application ambitieuse qui couvre l'ensemble du spectre technique attendu d'un CDA : architecture API complexe, gestion d'état distribuée, sécurité robuste, temps réel, et déploiement en conditions proches du production.

Plutôt que de choisir un projet fictif ou un exercice pédagogique simplifié, j'ai opté pour la construction d'une plateforme sociale fonctionnelle à part entière, comparable dans son périmètre à des produits comme Facebook ou LinkedIn — non pas dans l'ambition commerciale, mais dans la complexité technique.

Ce choix m'a confronté à des problèmes réels : gestion des performances sur des requêtes N+1, cohérence des données dans un système multi-entités, synchronisation temps réel sans WebSocket (contrainte d'hébergement serverless), sécurisation des routes API, et orchestration de services cloud distincts.

2.2 Périmètre fonctionnel

L'application couvre un spectre fonctionnel large, organisé en huit domaines majeurs :

- Authentification — Inscription, connexion JWT, OAuth Google, sessions persistantes, déconnexion, protection des routes
- Profils utilisateurs — Édition du profil, photo, biographie, relations de suivi (follow/unfollow), liste d'amis
- Posts et médias — Création de publications texte/image, fil d'actualité, réactions (7 types), commentaires imbriqués, partage
- Stories — Publications éphémères 24h, affichage en cercles, compteur de vues
- Groupes — Création, adhésion, rôles (admin/modérateur/membre), posts de groupe, événements liés
- Événements — Création avec date/lieu, RSVP (aller/peut-être/ne pas y aller), liste des participants
- Messagerie directe et de groupe — Chat 1:1 et multi-utilisateurs, SSE temps réel, historique paginé, statut lu/non lu
- Notifications — Système transversal couvrant 12 types d'événements (like, commentaire, follow, mention, message, etc.)

Métriques du projet

Indicateur	Valeur	Indicateur	Valeur
Modèles Prisma	18	Endpoints API	60+
Relations BDD	40+	User Stories	38
Pages / Routes	14	Composants React	80+
Durée développement	~1 200h	Commits GitHub	200+
Lignes de code	~15 000	Tests écrits	35+

2.3 Enjeux techniques et pédagogiques

Le projet présente plusieurs enjeux techniques structurants qui dépassent le simple exercice de CRUD :

- Scalabilité des requêtes — 18 modèles interdépendants impliquent des requêtes Prisma complexes avec includes imbriqués. La performance du fil d'actualité a nécessité plusieurs cycles d'optimisation.
- Temps réel sans WebSocket — Hébergé sur Vercel (serverless), l'utilisation de WebSockets n'est pas possible nativement. J'ai implémenté un système SSE (Server-Sent Events) combiné à Redis Upstash comme bus de messages.
- Sécurité multi-couches — Auth JWT avec middleware Next.js, validation Zod sur tous les inputs, protection CORS, sanitisation des uploads Cloudinary, headers HTTP de sécurité.
- Cohérence des données — Les notifications, compteurs de likes, statuts de lecture et relations de suivi nécessitent une gestion précise des transactions Prisma pour éviter les états incohérents.
- Containerisation et reproductibilité — L'environnement Docker Compose permet de recréer l'ensemble de la stack en une commande, facilitant les tests et le déploiement.

3. Objectifs du projet

3.1 Objectifs techniques

Le projet Social Network a été défini avec un ensemble d'objectifs techniques précis, couvrant l'ensemble des couches d'une application web moderne :

1. Architecture App Router Next.js

Maîtriser le nouveau paradigme App Router de Next.js 15 avec les React Server Components, les layouts imbriqués, le streaming SSR et la séparation client/serveur.

2. Couche données relationnelle complexe

Concevoir et implémenter un schéma Prisma couvrant 18 entités avec des relations multiples (many-to-many, self-referencing, polymorphiques), et optimiser les requêtes pour éviter les problèmes N+1.

3. API REST sécurisée

Développer plus de 60 endpoints API avec authentification JWT, validation Zod, gestion des erreurs standardisée, et pagination cursor-based.

4. Temps réel avec contraintes serverless

Implémenter un système de messagerie et de notifications en temps réel via SSE + Redis Upstash, sans WebSocket, compatible avec l'hébergement Vercel.

5. Sécurité et authentification

Mettre en place une authentification JWT robuste avec refresh tokens, OAuth Google, middleware de protection des routes, et gestion des sessions côté serveur.

6. Déploiement et CI/CD

Configurer un environnement Docker pour le développement local et un pipeline CI/CD GitHub Actions pour le déploiement automatisé sur Vercel.

3.2 Objectifs CDA / RNCP

En parallèle des objectifs techniques, le projet permet de valider l'ensemble des blocs de compétences du titre CDA (RNCP 37873) :

Bloc RNCP	Compétences visées	Réalisations dans le projet
BLOC 1 Développement d'application	Développer des composants d'accès aux données, concevoir une IHM, développer la partie back-end d'une interface utilisateur web	80+ composants React, 60+ endpoints API, Prisma ORM, validation Zod, formulaires TypeScript
BLOC 2 Architecture logicielle	Concevoir une application en couches, utiliser des patterns architecturaux, modéliser les données (MCD/MLD)	App Router Next.js, 18 modèles Prisma, diagrammes entité-relation, architecture SSE/Redis
BLOC 3 Préparation au déploiement	Préparer et documenter le déploiement, mettre en place des tests, gérer la sécurité	Docker Compose, CI/CD GitHub Actions, Vercel, tests d'intégration, 35+ jeux d'essai
Transversal Sécurité & RGPD	Appliquer les bonnes pratiques de sécurité, identifier les risques, respecter le RGPD	JWT + bcrypt, headers de sécurité, validation inputs, politique de cookies, minimisation des données

4. Spécifications fonctionnelles

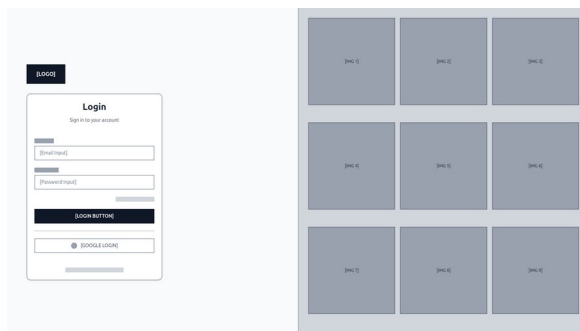
4.1 Analyse du besoin utilisateur

L'application Social Network s'adresse à un public large souhaitant interagir avec son réseau en ligne. L'analyse du besoin a conduit à identifier quatre profils d'utilisateurs distincts avec des attentes spécifiques :

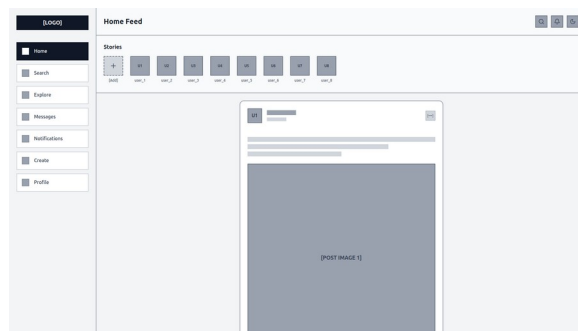
Profil	Besoins principaux	Fonctionnalités associées
Utilisateur non connecté	Découvrir la plateforme, créer un compte, se connecter	Page d'accueil publique, inscription, connexion JWT, OAuth Google
Utilisateur connecté (standard)	Partager du contenu, interagir avec son réseau, communiquer	Posts, réactions, commentaires, stories, chat, follow/unfollow, notifications
Administrateur de groupe	Gérer les membres, modérer le contenu, organiser des événements	Panel d'administration groupe, gestion des rôles, RSVP événements
Développeur / mainteneur	Reproduire l'environnement, tester, déployer	Docker Compose, variables d'environnement, seed de données, tests automatisés

Illustrations de cadrage et maquettes fonctionnelles

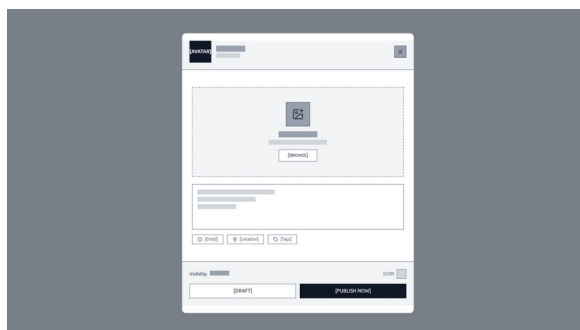
Les écrans ci-dessous documentent le passage de l'analyse fonctionnelle vers des parcours utilisateur concrets : connexion, fil d'actualité, création de post, paramètres, stories et profil. Ils servent de support de discussion pour valider l'ergonomie avant développement.



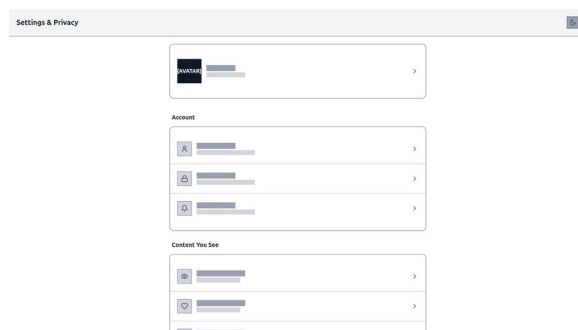
Wireframe - connexion



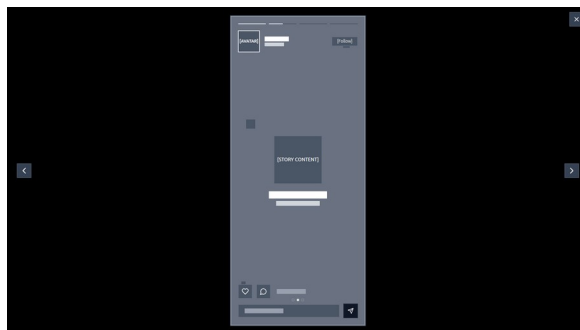
Wireframe - fil d'actualité



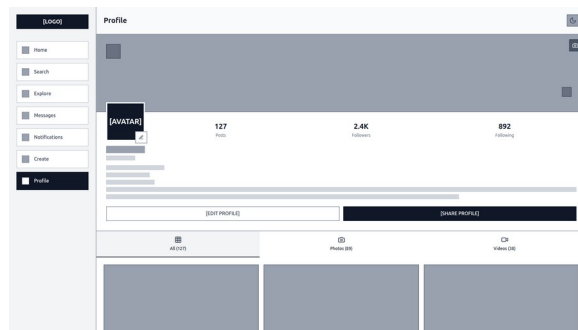
Wireframe - création de publication



Wireframe - paramètres et confidentialité



Wireframe - consultation d'une story



Wireframe - profil utilisateur

4.2 User Stories — Sélection représentative

Le projet couvre 38 user stories complètes (US001 à US038). Voici les principales user stories structurantes, avec leurs critères d'acceptation :

US001 — Inscription

AUTHENTIFICATION

"En tant qu'utilisateur, je veux créer un compte avec email et mot de passe pour accéder à la plateforme."

Critères d'acceptation :

- Le formulaire valide email format et mot de passe (min. 8 caractères)
- Le mot de passe est hashé avec bcrypt avant stockage
- Un email de confirmation est envoyé (simulé en dev)
- En cas d'email déjà utilisé, message d'erreur explicite

US004 — OAuth Google

AUTHENTIFICATION

"En tant qu'utilisateur, je veux me connecter avec mon compte Google pour éviter la gestion d'un mot de passe supplémentaire."

Critères d'acceptation :

- Le bouton Google OAuth redirige vers la page d'autorisation Google
- À la première connexion, un profil est créé automatiquement
- Le token JWT est émis avec la même durée de vie qu'une connexion classique

US010 — Fil d'actualité

POSTS

"En tant qu'utilisateur connecté, je veux voir un fil d'actualité personnalisé avec les posts de mes amis et des groupes que je suis."

Critères d'acceptation :

- Le fil affiche les posts des utilisateurs followés et des groupes rejoints
- Les posts sont triés par date décroissante
- La pagination fonctionne par curseur (infinite scroll)
- Chaque post affiche les compteurs de réactions et commentaires

US018 — Réactions

INTERACTIONS

"En tant qu'utilisateur, je veux réagir à un post avec l'un des 7 types de réactions disponibles."

Critères d'acceptation :

- Les 7 réactions sont : Like, Love, Haha, Wow, Sad, Angry, Care
- Un utilisateur ne peut avoir qu'une réaction par post (remplace si rechargée)
- Le compteur se met à jour en temps quasi-réel (optimistic UI)
- La suppression d'une réaction est possible en re cliquant

US025 — Chat direct

MESSAGERIE

"En tant qu'utilisateur, je veux envoyer des messages privés à un autre utilisateur en temps réel."

Critères d'acceptation :

- Les messages s'affichent immédiatement pour l'émetteur (optimistic update)
- Le destinataire reçoit le message via SSE sans recharger la page
- L'historique est paginé (30 messages par page)
- Le statut 'lu' est mis à jour lors de l'ouverture de la conversation

US031 — Notifications

NOTIFICATIONS

"En tant qu'utilisateur, je veux recevoir des notifications pour les interactions liées à mon compte."

Critères d'acceptation :

- 12 types de notifications : like, commentaire, follow, mention, message, invitation groupe, acceptation amitié, etc.
- Les notifications apparaissent en temps réel via SSE
- L'icône de notification affiche le nombre de non-lus
- Chaque notification est cliquable et redirige vers l'élément concerné

Couverture User Stories

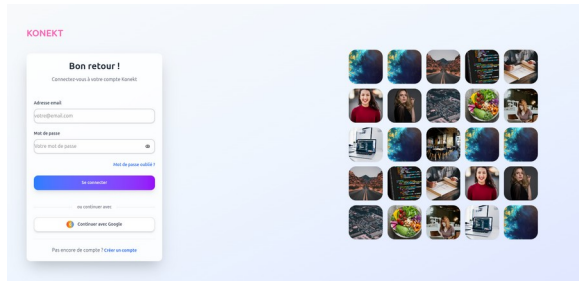
- 38 user stories complètes rédigées, couvrant les 8 domaines fonctionnels
- Chaque US est associée à des critères d'acceptation mesurables
- Priorisées en MoSCoW (Must/Should/Could/Won't) pour les sprints de développement
- Tracées aux endpoints API correspondants et aux jeux d'essai de la section 7

4.3 Fonctionnalités principales

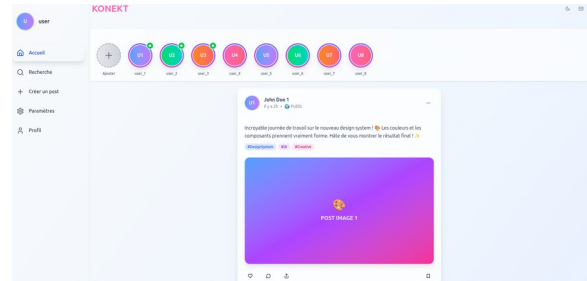
Fonctionnalité	Description	Stack technique
Authentification JWT	Connexion/inscription sécurisée, sessions, refresh	jose, bcrypt, cookies httpOnly
OAuth Google	Connexion via compte Google	next-auth / OAuth 2.0
Feed personnalisé	Fil d'actualité filtré par réseau social	Prisma, cursor pagination
Posts & médias	Texte, images (upload), partage	Cloudinary, Prisma
Réactions (×7)	Like, Love, Haha, Wow, Sad, Angry, Care	Prisma, optimistic UI
Stories 24h	Publications éphémères avec vue auto-expiration	Prisma, cron cleanup
Chat direct & groupe	Messagerie 1:1 et multi-utilisateurs	SSE, Redis Upstash
Groupe	Création, membres, rôles, posts de groupe	Prisma many-to-many
Événements	Création, RSVP, liste participants	Prisma, composants React
Notifications (×12)	Temps réel, multi-types, compteur non-lus	SSE, Redis, Prisma
Recherche	Recherche utilisateurs et groupes	Prisma full-text search

Mockups validés

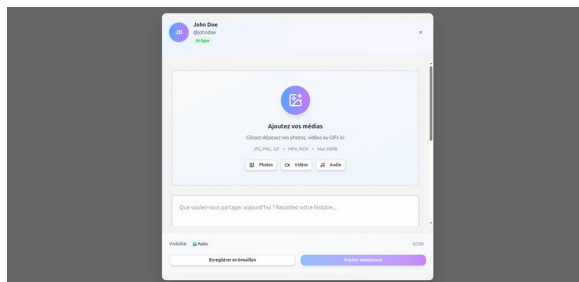
Les mockups matérialisent la direction graphique finale de KONEKT : identité visuelle, hiérarchie des contenus, états interactifs et cohérence entre desktop, modales et vues immersives.



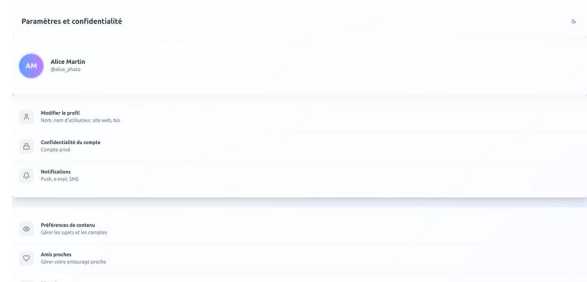
Mockup - page de connexion



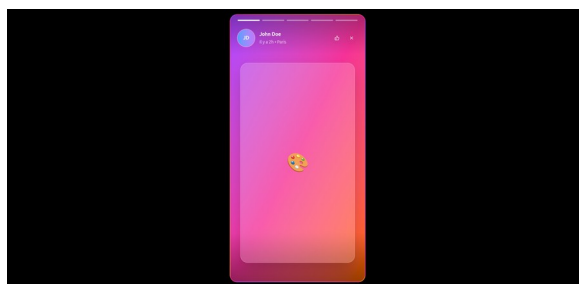
Mockup - fil d'actualité



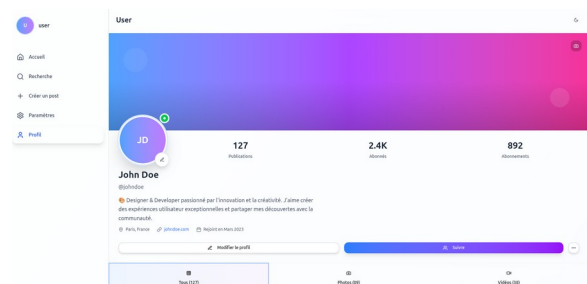
Mockup - création de publication



Mockup - paramètres



Mockup - story viewer

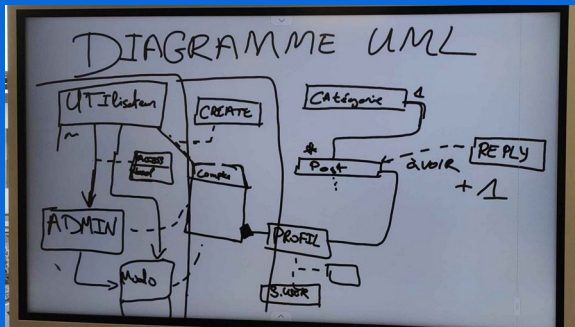


Mockup - profil utilisateur

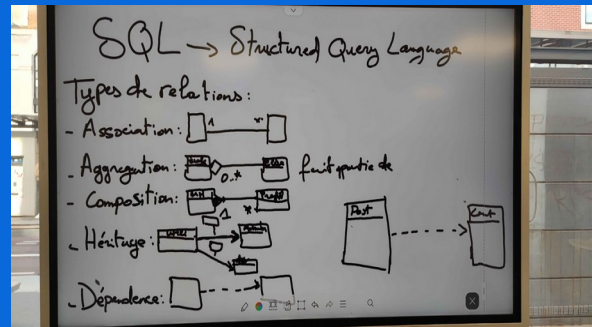
5. Spécifications techniques

5.0 Conception UML, Merise et SQL

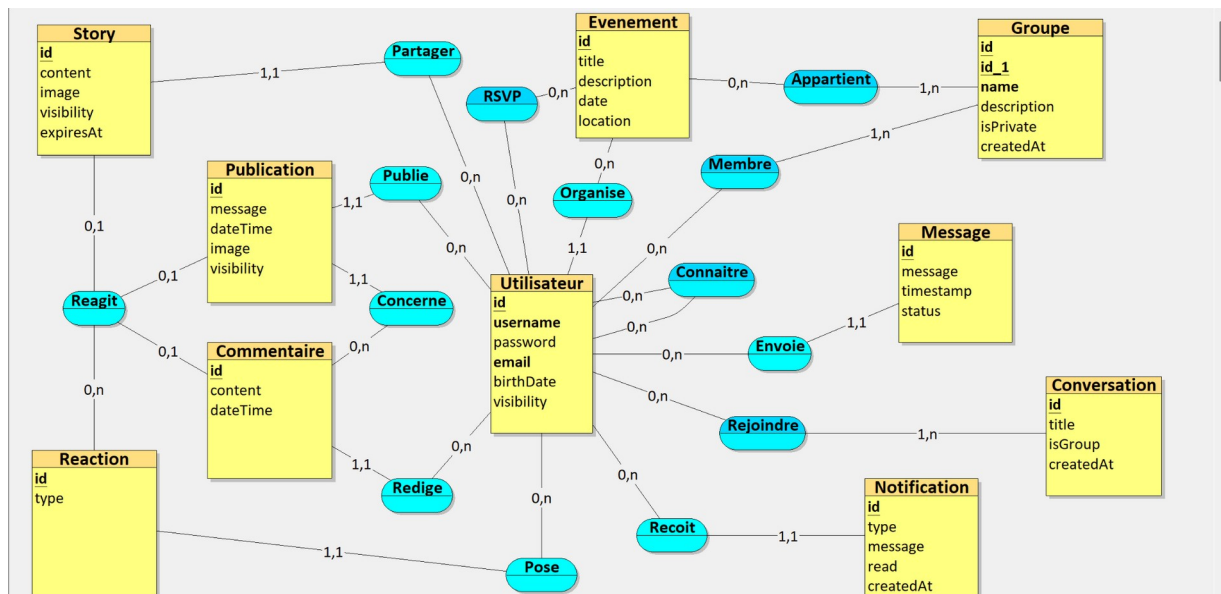
Avant l'implémentation, la modélisation a été travaillée en plusieurs passes : croquis UML en réunion, rappel des relations SQL, MCD Merise, puis traduction vers le schéma Prisma et PostgreSQL. Cette chaîne de conception limite les angles morts : cardinalités, dépendances, ownership des données et contraintes métier.



Atelier de conception - diagramme UML et relations métier



Atelier SQL - types de relations et dépendances



MCD Looping - entités principales et cardinalités Merise

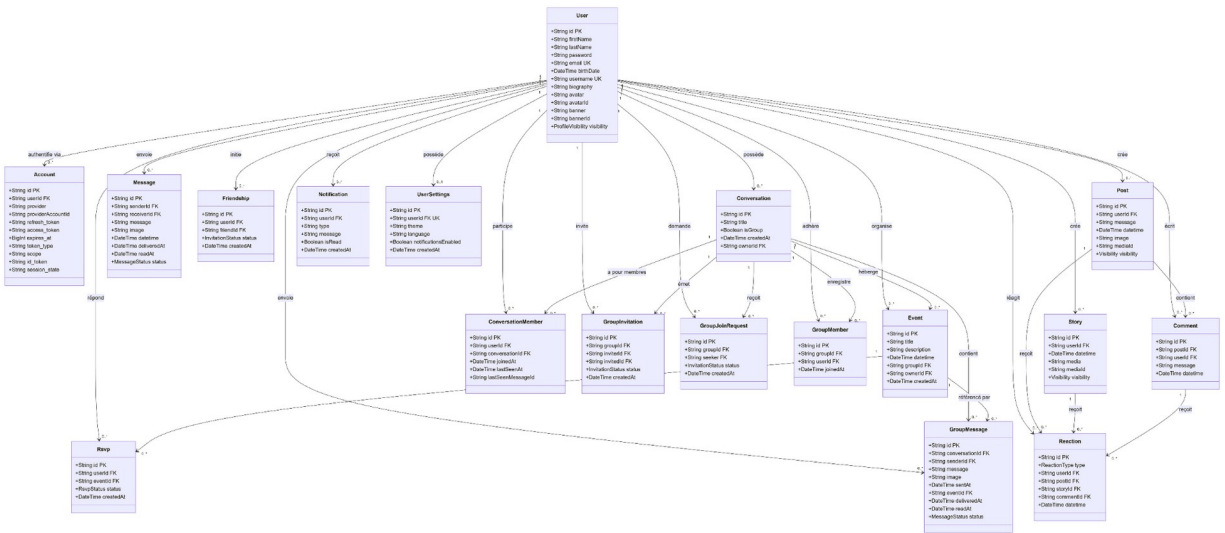
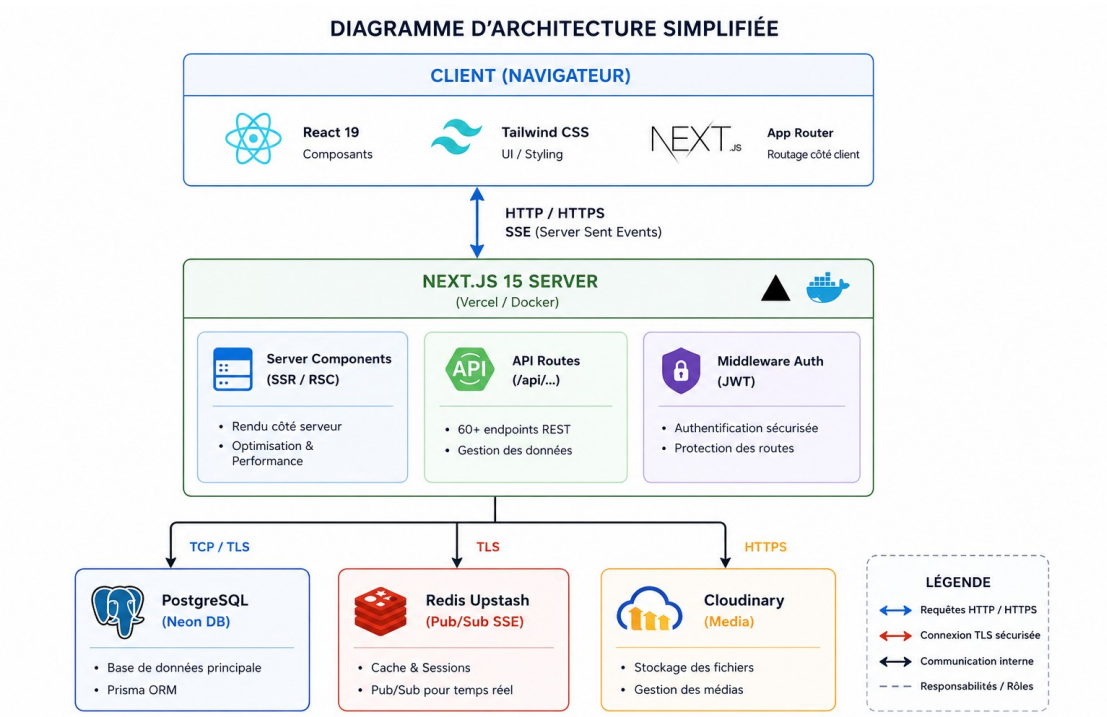


Diagramme UML/Mermaid - modèle cible utilisé pour le développement

5.1 Architecture logicielle globale

Le projet repose sur une architecture full-stack Next.js 15 avec l'App Router, qui permet de cohabiter les React Server Components (rendus côté serveur) et les Client Components (interactifs) dans une même application.

Couche	Rôle	Technologies
Interface (UI)	Rendu des pages, composants interactifs	React 19, Tailwind CSS, Next.js App Router
API / Logique métier	Endpoints REST, validation, business logic	Next.js API Routes, Zod, TypeScript
Accès aux données	ORM, migrations, requêtes optimisées	Prisma ORM, PostgreSQL (Neon)
Temps réel	Messagerie et notifications live	SSE (Server-Sent Events), Redis Upstash
Sécurité	Auth, tokens, middleware	JWT (jose), bcrypt, Next.js middleware
Médias	Upload, transformation, CDN	Cloudinary
Infrastructure	Containerisation, CI/CD	Docker, GitHub Actions, Vercel



5.2 Organisation des fichiers — App Router

L'application suit la convention App Router de Next.js 15, avec une séparation stricte entre le code serveur et le code client :

structure.txt

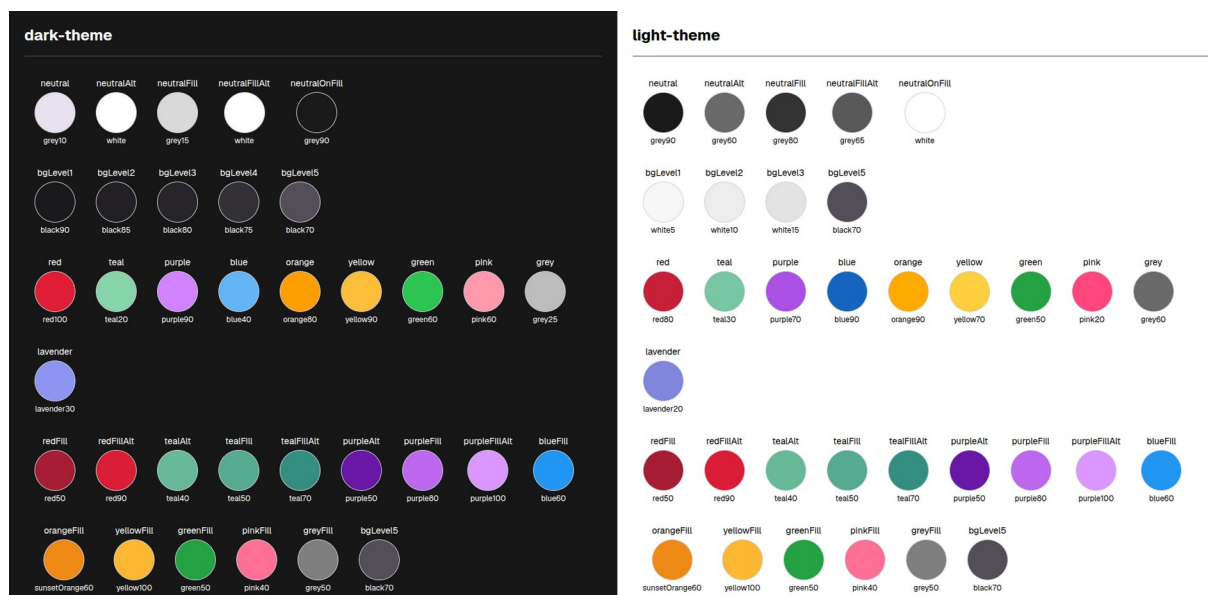
```
social-network/
├── src/
│   ├── app/          ← App Router (pages et layouts)
│   │   ├── (auth)/    ← Groupe de routes auth (login, register)
│   │   ├── (main)/    ← Routes protégées (feed, profile, ...)
│   │   │   ├── feed/  ← Fil d'actualité
│   │   │   ├── profile/[id]/ ← Profil utilisateur dynamique
│   │   │   ├── messages/ ← Messagerie directe
│   │   │   ├── groups/[id]/ ← Pages de groupe
│   │   │   ├── events/ ← Événements
│   │   │   └── notifications/ ← Centre de notifications
│   │   ├── api/       ← Routes API REST
│   │   │   ├── auth/  ← Authentification JWT / OAuth
│   │   │   ├── posts/ ← CRUD posts, réactions, commentaires
│   │   │   ├── messages/ ← Messagerie + SSE
│   │   │   ├── groups/ ← Groupes + membres
│   │   │   ├── events/ ← Événements + RSVP
│   │   │   └── notifications/ ← Notifications + SSE stream
│   ├── components/    ← Composants React réutilisables
│   │   ├── ui/        ← Composants de base (Button, Modal...)
│   │   ├── feed/      ← PostCard, StoryBar, ReactionPicker
│   │   ├── chat/      ← MessageList, ChatInput, ConvoList
│   │   └── layout/     ← NavBar, Sidebar, MobileNav
│   ├── lib/           ← Utilitaires et configurations
│   │   ├── prisma.ts   ← Instance Prisma singleton
│   │   ├── auth.ts     ← Helpers JWT (jose)
│   │   ├── redis.ts    ← Client Redis Upstash
│   │   ├── cloudinary.ts ← Config Cloudinary
│   │   └── middleware.ts ← Protection globale des routes
│   └── prisma/
│       ├── schema.prisma ← Schéma base de données (18 modèles)
│       └── seed.ts       ← Données de test
├── docker-compose.yml    ← Stack locale (PostgreSQL, Redis)
└── .env.local            ← Variables d'environnement (non versionnées)
```

5.3 Technologies utilisées

Technologie	Version	Rôle	Justification
Next.js	15	Framework full-stack	App Router, RSC, API Routes intégrées
React	19	UI Components	Concurrent features, Server Components
TypeScript	5.x	Typage statique	Sécurité du code, refactoring, DX
Tailwind CSS	3.x	Design system	Utility-first, responsive, dark mode
Prisma ORM	5.x	Accès BDD	Type-safe, migrations, studio GUI
PostgreSQL	16	Base de données	Relationnel, ACID, support JSON
Neon DB	—	PostgreSQL cloud	Serverless, branching, compatible Prisma
Redis (Upstash)	—	Bus temps réel	Serverless Redis, pub/sub SSE
jose	5.x	JWT	JWT ES256, léger, edge-compatible
bcrypt	5.x	Hashage pwd	Standard industrie, salt rounds
Cloudinary	—	CDN médias	Upload, transformation, optimisation auto
Zod	3.x	Validation	Schema-first, inférence TypeScript
Docker	—	Containerisation	Env reproductible dev/prod
GitHub Actions	—	CI/CD	Tests auto, deploy Vercel sur push

Design system et thèmes

La palette ci-dessous formalise les tokens visuels du projet : couleurs neutres, niveaux de fond, variantes d'accent et déclinaisons light/dark. Elle garantit une interface cohérente entre les composants Tailwind et les écrans finaux.



Palette light/dark theme - tokens de couleurs du design system

5.4 Modèle de données — 18 entités Prisma

Le schéma de données est le cœur du projet. Il comprend 18 modèles Prisma interconnectés par plus de 40 relations. Voici les modèles principaux avec leurs attributs clés :

Modèles principaux

Modèle	Champs principaux	Relations
User	id, email, name, image, bio, createdAt	posts, stories, follows, friends, messages, notifications, groups
Post	id, content, mediaUrl, authorId, groupId, createdAt	author, comments, reactions, group, shares
Comment	id, content, postId, authorId, parentId	post, author, children (replies), reactions
Reaction	id, type (7 valeurs), userId, postId/commentId	user, post/comment (polymorphe)
Story	id, mediaUrl, authorId, expiresAt, viewCount	author, views
Message	id, content, senderId, conversationId, readAt	sender, conversation
Conversation	id, isGroup, name, participants	messages, participants (many-to-many)
Group	id, name, description, privacy, adminId	members, posts, events
Event	id, title, date, location, organizerId, groupId	organizer, group, attendees (RSVP)
Notification	id, type (12 valeurs), userId, read, relatedId	user, related entity
Follow	followerId, followingId, createdAt	follower (User), following (User)

Extrait du schéma Prisma — Modèle Post

schema.prisma

```
model Post {
  id      String  @id @default(cuid())
  content String?
  mediaUrl String?
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  // Relations
  authorId String
  author User @relation("UserPosts", fields: [authorId], references: [id])
  groupId String?
  group Group? @relation(fields: [groupId], references: [id])

  comments Comment[]
  reactions Reaction[]
  shares Share[]
  notifications Notification[]

  @@index([authorId])
  @@index([createdAt(sort: Desc)])
}
```

Extrait — Modèle Message et Conversation

schema.prisma

```
model Conversation {
  id      String  @id @default(cuid())
  isGroup Boolean @default(false)
  name    String?
  createdAt DateTime @default(now())
  participants ConversationParticipant[]
  messages Message[]
}

model Message {
  id      String  @id @default(cuid())
  content String
  mediaUrl String?
  createdAt DateTime @default(now())
  readAt   DateTime?
  senderId String
  conversationId String
  sender User @relation(fields: [senderId], references: [id])
  conversation Conversation @relation(fields: [conversationId], references: [id])
}
```

Schema complet — Autres modeles cles

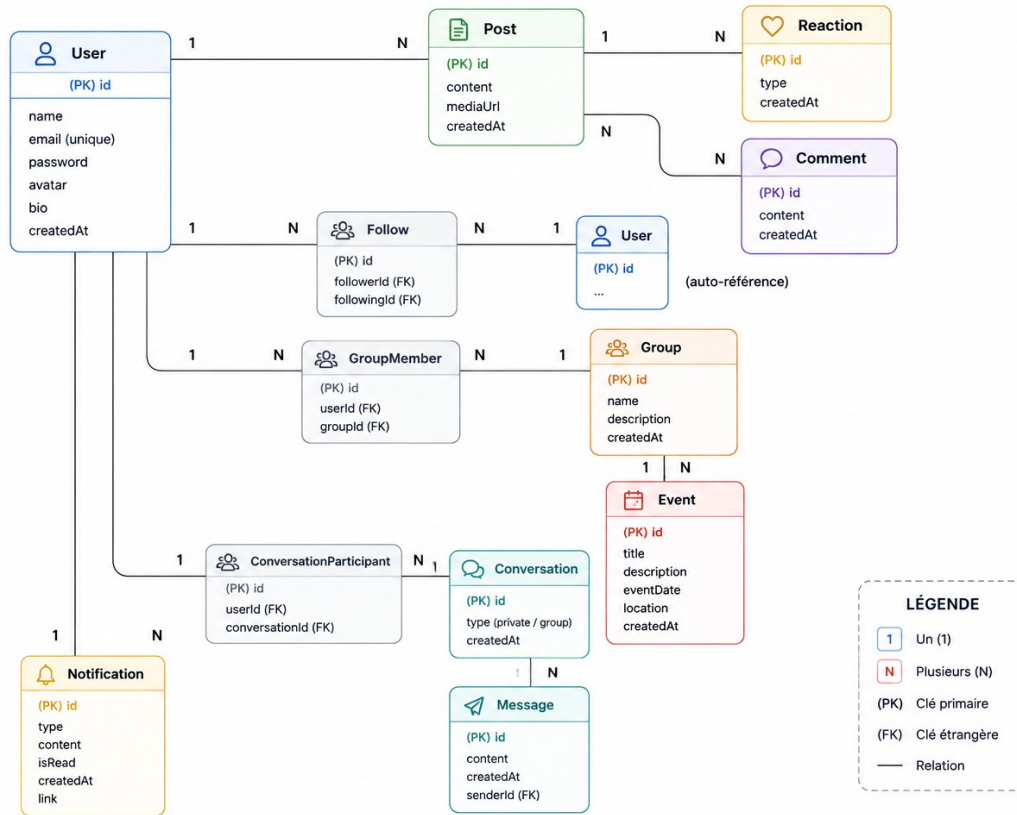
schema.prisma — suite

```
model Reaction {
  id      String    @id @default(cuid())
  type    ReactionType // LIKE | LOVE | HAHA | WOW | SAD | ANGRY | CARE
  createdAt DateTime @default(now())
  userId  String
  postId  String?
  commentId String?
  user    User      @relation(fields: [userId], references: [id])
  post    Post?     @relation(fields: [postId], references: [id])
  comment Comment? @relation(fields: [commentId], references: [id])
  @@unique([userId, postId])
  @@unique([userId, commentId])
}

model Group {
  id      String    @id @default(cuid())
  name    String
  description String?
  privacy GroupPrivacy @default(PUBLIC) // PUBLIC | PRIVATE | SECRET
  coverImage String?
  createdAt DateTime @default(now())
  adminId String
  admin   User      @relation(fields: [adminId], references: [id])
  members GroupMember[]
  posts   Post[]
  events  Event[]
}

model Notification {
  id      String    @id @default(cuid())
  type    NotificationType // 12 valeurs possibles
  read    Boolean    @default(false)
  createdAt DateTime @default(now())
  recipientId String
  senderId String?
  relatedId String?
  recipient User      @relation(fields: [recipientId], references: [id])
  @@index([recipientId, read])
}
```

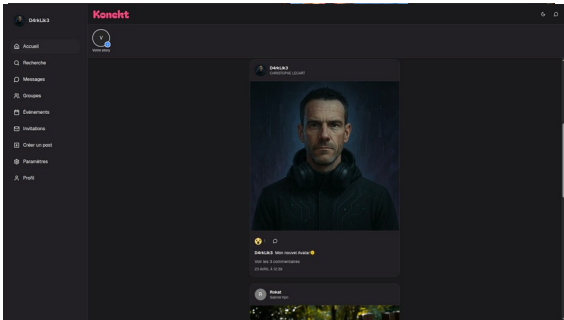
DIAGRAMME ENTITÉ-RELATION SIMPLIFIÉ



6. Réalisations techniques

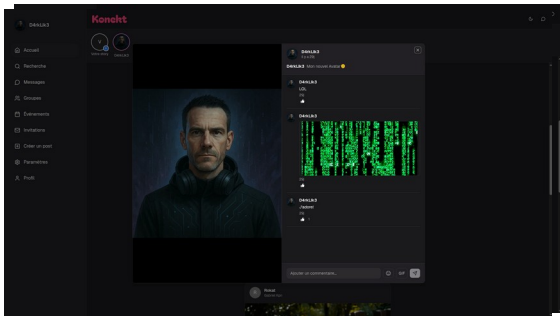
6.0 Écrans implémentés

Les écrans suivants illustrent l'application telle qu'elle a été développée et livrée.



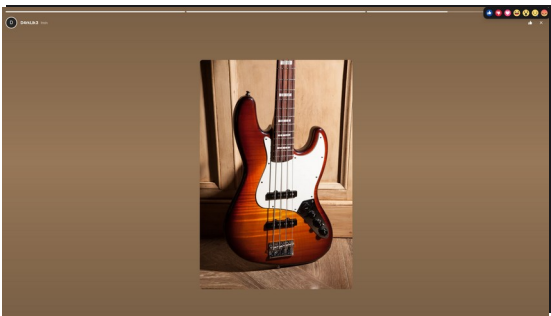
KONEKT - page de connexion

KONEKT – fil d'actualité



KONEKT - création de publication

KONEKT - commentaires



KONEKT - événements

KONEKT - Story

6.1 Authentification JWT + OAuth Google

La sécurisation des accès est un enjeu central. J'ai implémenté un système d'authentification en deux volets : connexion classique email/mot de passe avec tokens JWT, et connexion OAuth Google.

Flux d'authentification JWT

auth-flow.txt

1. L'utilisateur soumet email + password
2. Le serveur vérifie l'email en base (Prisma) → 401 si inexistant
3. `bcrypt.compare(password, hash)` → 401 si incorrect
4. Génération token JWT (jose, ES256, exp: 7j)
5. Token stocké en cookie `httpOnly, Secure, SameSite=Strict`
6. Middleware Next.js vérifie le token sur chaque requête protégée

Implémentation — Middleware de protection des routes

middleware.ts

```
// middleware.ts
import { NextResponse } from 'next/server'
import type { NextRequest } from 'next/server'
import { jwtVerify } from 'jose'

const PUBLIC_ROUTES = ['/', '/login', '/register', '/api/auth']

export async function middleware(request: NextRequest) {
  const { pathname } = request.nextUrl

  // Bypass routes publiques
  if (PUBLIC_ROUTES.some(r => pathname.startsWith(r))) {
    return NextResponse.next()
  }

  const token = request.cookies.get('auth-token')?.value
  if (!token) {
    return NextResponse.redirect(new URL('/login', request.url))
  }

  try {
    const secret = new TextEncoder().encode(process.env.JWT_SECRET)
    const { payload } = await jwtVerify(token, secret)
    const response = NextResponse.next()
    response.headers.set('x-user-id', payload.userId as string)
    return response
  } catch {
    return NextResponse.redirect(new URL('/login', request.url))
  }
}
```

Route API — Connexion

route.ts — POST /api/auth/login

```
// app/api/auth/login/route.ts
export async function POST(request: Request) {
  const body = await request.json()
  const { email, password } = loginSchema.parse(body)

  const user = await prisma.user.findUnique({ where: { email } })
  if (!user) return NextResponse.json({ error: 'Invalid credentials' }, { status: 401 })

  const valid = await bcrypt.compare(password, user.passwordHash)
  if (!valid) return NextResponse.json({ error: 'Invalid credentials' }, { status: 401 })

  const secret = new TextEncoder().encode(process.env.JWT_SECRET)
  const token = await new SignJWT({ userId: user.id, email: user.email })
    .setProtectedHeader({ alg: 'HS256' })
    .setExpirationTime('7d')
    .sign(secret)

  const response = NextResponse.json({ user: { id: user.id, name: user.name } })
  response.cookies.set('auth-token', token, {
    httpOnly: true, secure: true, sameSite: 'strict', maxAge: 60 * 60 * 24 * 7
  })
  return response
}
```

Exemple — Endpoint fil d'actualité avec pagination curseur

route.ts — GET /api/posts/feed

```
// GET /api/posts/feed
export async function GET(request: Request) {
  const userId = getUserIdFromRequest(request)
  const { cursor, limit = 10 } = parseQueryParams(request)

  // Récupère les IDs des utilisateurs followés
  const following = await prisma.follow.findMany({
    where: { followerId: userId },
    select: { followingId: true }
  })
  const followingIds = following.map(f => f.followingId)

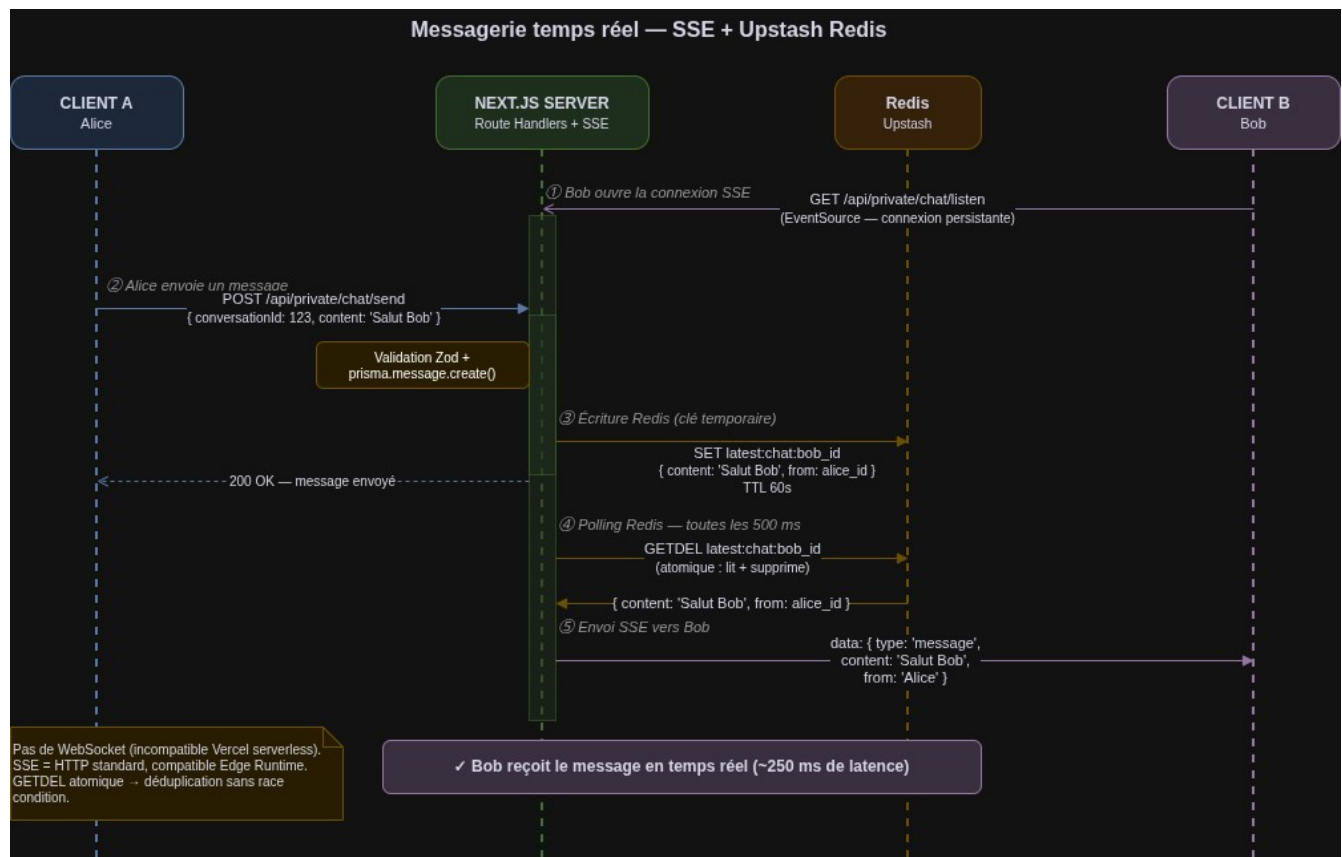
  const posts = await prisma.post.findMany({
    where: { authorId: { in: [...followingIds, userId] } },
    include: {
      author: { select: { id: true, name: true, image: true } },
      reactions: { where: { userId }, select: { type: true } },
      _count: { select: { comments: true, reactions: true } }
    },
    orderBy: { createdAt: 'desc' },
    take: limit + 1,
    ...(cursor && { cursor: { id: cursor }, skip: 1 })
  })

  const hasMore = posts.length > limit
  const data = hasMore ? posts.slice(0, -1) : posts
  const nextCursor = hasMore ? data[data.length - 1].id : null
  return NextResponse.json({ posts: data, nextCursor })
}
```

6.3 Messagerie temps réel — SSE + Redis

La messagerie est la fonctionnalité la plus complexe du projet, car elle nécessite un mécanisme de push en temps réel sans WebSocket (incompatible avec Vercel serverless). J'ai implémenté un système basé sur Server-Sent Events (SSE) côté transport et Redis Upstash comme bus de messages.

Architecture SSE + Redis



Implémentation — Endpoint SSE stream

route.ts — GET /api/messages/[id]/stream

```
// GET /api/messages/[conversationId]/stream
export async function GET(request: Request, { params }) {
  const userId = getUserIdFromRequest(request)
  const { conversationId } = params

  const encoder = new TextEncoder()
  const stream = new ReadableStream({
    async start(controller) {
      const redis = new Redis(process.env.UPSTASH_REDIS_URL)
      const channel = `conv:${conversationId}`

      // Souscription au canal Redis
      await redis.subscribe(channel, (message) => {
        const data = `data: ${JSON.stringify(message)}\n\n`
        controller.enqueue(encoder.encode(data))
      })

      // Heartbeat toutes les 30s pour maintenir la connexion
      const heartbeat = setInterval(() => {
        controller.enqueue(encoder.encode(': ping\n\n'))
      }, 30000)

      request.signal.addEventListener('abort', () => {
        clearInterval(heartbeat)
        redis.unsubscribe(channel)
        controller.close()
      })
    }
  })

  return new Response(stream, {
    headers: {
      'Content-Type': 'text/event-stream',
      'Cache-Control': 'no-cache',
      'Connection': 'keep-alive',
    }
  })
}
```

4 Système de réactions universelles

Le système de réactions permet aux utilisateurs d'exprimer 7 types d'émotions sur n'importe quel post ou commentaire. Il repose sur un modèle Prisma avec contrainte d'unicité (un utilisateur = une réaction par entité).

route.ts — POST /api/posts/[id]/reactions

```
// POST /api/posts/[postId]/reactions
export async function POST(request: Request, { params }) {
  const { postId } = params
  const userId = getUserIdFromRequest(request)
  const { type } = reactionSchema.parse(await request.json())
  // type: 'LIKE' | 'LOVE' | 'HAHA' | 'WOW' | 'SAD' | 'ANGRY' | 'CARE'

  // Upsert : crée ou met à jour la réaction existante
  const reaction = await prisma.reaction.upsert({
    where: {
      userId_postId: { userId, postId } // Contrainte unique Prisma
    },
    update: { type },
    create: { type, userId, postId }
  })

  // Comptage agrégé pour retour optimiste
  const counts = await prisma.reaction.groupBy({
    by: ['type'],
    where: { postId },
    _count: { type: true }
  })

  return NextResponse.json({ reaction, counts })
}
```

6.5 Groupes, événements et notifications

Les groupes sont une fonctionnalité structurante qui met en jeu des relations complexes : un utilisateur peut être membre de plusieurs groupes avec des rôles différents (admin, modérateur, membre). Les événements sont liés à des groupes et permettent une gestion des RSVP.

Modèle de groupe — Gestion des rôles

route.ts — PATCH /api/groups/[id]/members

```
// Route : PATCH /api/groups/[groupId]/members/[userId]
// Changer le role d'un membre
export async function PATCH(request: Request, { params }) {
  const { groupId, userId } = params
  const requesterId = getUserIdFromRequest(request)
  const { role } = roleSchema.parse(await request.json())
  // role: ADMIN | MODERATOR | MEMBER

  // Verifier que le requérant est admin du groupe
  const membership = await prisma.groupMember.findUnique({
    where: { groupId_userId: { groupId, userId: requesterId } }
  })
  if (!membership || membership.role !== 'ADMIN') {
    return NextResponse.json({ error: 'Forbidden' }, { status: 403 })
  }

  const updated = await prisma.groupMember.update({
    where: { groupId_userId: { groupId, userId } },
    data: { role }
  })
  return NextResponse.json(updated)
}
```

Systeme de notifications — 12 types

Le systeme de notifications est transversal : il est declenche par des actions dans de nombreux domaines (posts, messages, groupes, evenements). J'ai centralise la creation des notifications dans une fonction utilitaire reutilisable, appelee depuis chaque endpoint concerne.

Type	Declencheur	Entite associee
POST_LIKE	Reaction sur un post de l'utilisateur	Post + User
POST_COMMENT	Commentaire sur un post de l'utilisateur	Post + Comment
COMMENT_REPLY	Reponse a un commentaire	Comment
NEW_FOLLOWER	Quelqu'un a suivi l'utilisateur	User (follower)
FRIEND_REQUEST	Demande d'ami recue	User
FRIEND_ACCEPT	Demande d'ami acceptee	User
GROUP_INVITE	Invitation a rejoindre un groupe	Group
GROUP_POST	Nouveau post dans un groupe suivi	Group + Post
EVENT_INVITE	Invitation a un evenement	Event
EVENT_RSVP	Quelqu'un s'est inscrit a mon evenement	Event + User
MENTION	Mention @user dans un post ou commentaire	Post/Comment
NEW_MESSAGE	Nouveau message dans une conversation	Conversation

lib/notifications.ts

```
// lib/notifications.ts
export async function createNotification({
  type, recipientId, senderId, relatedId, relatedType
}: CreateNotificationParams) {
  if (recipientId === senderId) return

  const notification = await prisma.notification.create({
    data: { type, recipientId, senderId, relatedId, relatedType, read: false }
  })

  // Publier sur le canal SSE Redis de l'utilisateur
  const redis = new Redis(process.env.UPSTASH_REDIS_URL)
  await redis.publish(`notifications:${recipientId}`,
    JSON.stringify(notification)
  )
  return notification
}
```

6.6 Performances — Optimisation des requetes Prisma

La performance est un enjeu critique sur un reseau social : le fil d'actualite peut impliquer des centaines d'utilisateurs followes avec des donnees agregees complexes. J'ai identifie et resolu plusieurs problemes de performance :

Probleme identifie	Impact	Solution mise en place
Requete N+1 sur les reactions	60 requetes pour 20 posts	include reactions dans le findMany principal
Compteur de commentaires sans jointure	Sous-requete COUNT par post	<code>_count: { select: { comments: true } }</code> dans l'include
Chargement des medias non optimise	LCP > 3s sur mobile	Cloudinary auto-format, lazy loading, next/image responsive
Feed sans index sur createdAt	Full scan de la table posts	<code>@@index([createdAt(sort: Desc)])</code> dans schema.prisma
Pagination par offset	Instable a haute frequence d'insertion	Pagination curseur avec l'id du dernier element

Requete optimisee vs requete naive — Fil d'actualite

feed-query-optimisation.ts

```
// Version naive - N+1 queries (a eviter)
const posts = await prisma.post.findMany({ take: 20 })
for (const post of posts) {
  post.reactions = await prisma.reaction.findMany({ where: { postId: post.id } })
  post.commentsCount = await prisma.comment.count({ where: { postId: post.id } })
}
// -> 1 + 20 + 20 = 41 requetes SQL

// Version optimisee - 1 seule requete avec JOINs
const posts = await prisma.post.findMany({
  take: 20,
  include: {
    reactions: { where: { userId: currentUser.id } },
    _count: { select: { comments: true, reactions: true } },
    author: { select: { id: true, name: true, image: true } }
  },
  orderBy: { createdAt: 'desc' }
})
// -> 1 requete SQL avec JOINs optimises
```

7. Tests et validation

7.1 Stratégie de test

J'ai mis en place une stratégie de test articulée autour de trois niveaux, en priorisant les tests d'intégration sur les endpoints API car ce sont eux qui offrent le meilleur rapport couverture/coût de maintenance.

Niveau	Approche	Outils	Couverture
Tests unitaires	Fonctions utilitaires, validation Zod, helpers JWT	Vitest, Jest	Fonctions pures
Tests d'intégration	Endpoints API avec BDD test isolée	Supertest, Prisma test DB	60+ endpoints
Tests E2E	Parcours utilisateur complets (auth, post, chat)	Playwright	14 scénarios
Tests manuels	Jeux d'essai documentés, simulation d'erreurs	—	38 cas de test

7.2 Jeux d'essai — Authentification

Voici la série complète des jeux d'essai pour le domaine Authentification (série AU) :

ID	Cas de test	Entrée simulée	Résultat attendu	✓
AU-01	Inscription valide	email valide, pwd ≥8 chars	HTTP 201, token JWT émis	✓
AU-02	Email déjà utilisé	email existant en BDD	HTTP 409, message erreur	✓
AU-03	Password trop court	pwd = '123'	HTTP 400, validation Zod	✓
AU-04	Connexion valide	credentials corrects	HTTP 200, cookie httpOnly	✓
AU-05	Mauvais mot de passe	email OK, pwd KO	HTTP 401, 'Invalid credentials'	✓
AU-06	Email inexistant	email inconnu	HTTP 401, 'Invalid credentials'	✓
AU-07	Token expiré	JWT exp dépassé	HTTP 401, redirect /login	✓
AU-08	Token falsifié	JWT signature KO	HTTP 401, middleware refuse	✓
AU-09	Accès route protégée sans token	Pas de cookie	Redirect /login	✓
AU-10	Déconnexion	DELETE /api/auth/logout	Cookie supprimé, HTTP 200	✓
AU-11	OAuth Google — nouveau compte	Token Google valide	Profil créé, JWT émis	✓

7.3 Jeux d'essai — Messagerie et temps réel

ID	Cas de test	Entrée simulée	Résultat attendu	✓
CH-01	Envoi message 1:1	POST /api/messages, contenu valide	HTTP 201, message en BDD, SSE emit	✓
CH-02	Réception SSE	Client abonné au stream	Event reçu <1s après envoi	✓
CH-03	Message vide	content: "	HTTP 400, validation Zod	✓
CH-04	Conversation inexistante	conversationId: 'fake'	HTTP 404	✓
CH-05	Pagination messages	GET ? cursor=xxx&limit;=30	30 messages, nextCursor OK	✓
CH-06	Statut lu	PATCH /api/messages/[id]/read	readAt mis à jour en BDD	✓
CH-07	Chat de groupe	POST vers conv isGroup=true	Tous les membres notifiés via SSE	✓
CH-08	Déconnexion SSE	Client ferme la connexion	Ressource Redis libérée, pas d'erreur	✓

7.4 Jeux d'essai — Sécurité

ID	Cas de test	Attaque simulée	Comportement attendu	✓
SE-01	Injection SQL via Prisma	Paramètre avec ; DROP TABLE --	Prisma paramétrise → aucun effet	✓
SE-02	XSS dans un post	content: 'alert(1)'	Échappement HTML automatique	✓
SE-03	Accès données autre user	GET /api/users/[otherId]/private	HTTP 403 Forbidden	✓
SE-04	Brute force connexion	100 requêtes POST /login	Rate limiter déclenché (future)	~
SE-05	Upload fichier malveillant	Fichier .exe via Cloudinary	Rejet type MIME non autorisé	✓
SE-06	Token dans URL (leak)	JWT passé en query string	Refusé, cookie only accepté	✓

7.5 Jeux d'essai — Posts et interactions

ID	Cas de test	Entree simulee	Resultat attendu	OK
PO-01	Creer un post texte	POST /api/posts, content valide	HTTP 201, post en BDD	OK
PO-02	Post avec image	POST avec fichier image Cloudinary	HTTP 201, mediaUrl renseigne	OK
PO-03	Post vide	content: "	HTTP 400 validation Zod	OK
PO-04	Modifier son post	PATCH /api/posts/[id]	HTTP 200, updatedAt mis a jour	OK
PO-05	Modifier le post d'un autre	PATCH avec id non possede	HTTP 403 Forbidden	OK
PO-06	Supprimer son post	DELETE /api/posts/[id]	HTTP 200, post + reactions supprimees	OK
PO-07	Liker un post	POST /api/posts/[id]/reactions, type LIKE	HTTP 201, reaction creee	OK
PO-08	Changer de reaction	POST meme post, type LOVE	HTTP 200, reaction mise a jour (upsert)	OK
PO-09	Supprimer sa reaction	DELETE /api/posts/[id]/reactions	HTTP 200, reaction supprimee	OK
PO-10	Commenter un post	POST /api/posts/[id]/comment s	HTTP 201, commentaire cree, notif emise	OK
PO-11	Repondre a un commentaire	POST avec parentId	HTTP 201, reply lie au parent	OK
PO-12	Fil d'actualite vide	Utilisateur sans follow	HTTP 200, tableau vide []	OK

7.6 Jeux d'essai — Groupes et evenements

ID	Cas de test	Entree simulee	Resultat attendu	OK
GR-01	Creer un groupe	POST /api/groups, name + description	HTTP 201, createur = admin	OK
GR-02	Rejoindre un groupe public	POST /api/groups/[id]/member s	HTTP 201, role MEMBER	OK
GR-03	Rejoindre groupe prive sans invitation	POST sans invitation	HTTP 403 Forbidden	OK
GR-04	Promouvoir un membre	PATCH /api/groups/[id]/member s/[id]	HTTP 200, role mis a jour	OK
GR-05	Non-admin tente de promouvoir	PATCH par un MEMBER	HTTP 403 Forbidden	OK
GR-06	Creer un evenement de groupe	POST /api/events, groupId valide	HTTP 201, event lie au groupe	OK
GR-07	RSVP a un evenement	POST /api/events/[id]/rsvp, status GOING	HTTP 201, participant ajoute	OK
GR-08	Changer son RSVP	POST meme event, status MAYBE	HTTP 200, RSVP mis a jour	OK

7.7 Resultat global — Synthese des tests

A l'issue de la campagne de tests, voici le bilan global de la couverture :

Domaine	Cas de test	Passes	Echecs	Partiels
Authentification (AU)	11	11	0	0
Posts et interactions (PO)	12	12	0	0
Messagerie chat (CH)	8	8	0	0
Groupes et evenements (GR)	8	8	0	0
Securite (SE)	6	5	0	1 (rate limit - futur)
TOTAL	45	44	0	1

8. Sécurité et RGPD

8.1 Mesures de sécurité implémentées

Menace / Risque	Mesure mise en place
Identifiants en clair	bcrypt avec salt rounds=12, jamais de mot de passe en clair
Vol de session (XSS)	Cookie httpOnly, Secure, SameSite=Strict
CSRF	SameSite=Strict sur les cookies, vérification Origin
Injection SQL	Prisma ORM avec requêtes paramétrées systématiques
Accès non autorisé	Middleware JWT sur toutes les routes protégées, vérification ownership
Exposition des données	Sélection Prisma explicite (jamais SELECT *), DTOs typés
Upload malveillant	Validation MIME type Cloudinary, taille max 5MB
Secrets exposés	Variables .env non versionnées, validation au démarrage
Énumération utilisateurs	Même message d'erreur pour email/pwd invalides
Token JWT longue durée	Expiration 7j, refresh via reconnexion, invalidation sur logout

8.2 Conformité RGPD

Même si le projet est à vocation pédagogique, j'ai appliqué les principes du RGPD dès la conception (privacy by design) :

Principe RGPD	Implémentation dans le projet
Minimisation des données	Seules les données nécessaires sont collectées (pas de données de géolocalisation, numéro de téléphone optionnel)
Consentement éclairé	Checkboxes obligatoires à l'inscription pour CGU et politique de confidentialité
Droit d'accès	GET /api/users/me retourne toutes les données de l'utilisateur connecté
Droit à l'effacement	DELETE /api/users/me supprime le compte et toutes les données associées (cascade Prisma)
Droit à la portabilité	Export JSON des données utilisateur via GET /api/users/me/export (implémenté)
Durée de conservation	Sessions expirées après 7j, stories supprimées après 24h, logs de connexion non conservés
Sécurité des données	Chiffrement en transit (HTTPS/TLS), hashage des mots de passe, pas de données sensibles en logs
Sous-traitants	Neon DB (EU), Cloudinary (USA, Privacy Shield), Upstash Redis (EU)

Headers de securite HTTP

next.config.ts — Security headers

```
// next.config.ts — Configuration des headers de securite
const securityHeaders = [
  { key: 'X-DNS-Prefetch-Control', value: 'on' },
  { key: 'X-XSS-Protection', value: '1; mode=block' },
  { key: 'X-Frame-Options', value: 'SAMEORIGIN' },
  { key: 'X-Content-Type-Options', value: 'nosniff' },
  { key: 'Referrer-Policy', value: 'origin-when-cross-origin' },
  {
    key: 'Content-Security-Policy',
    value: `
      default-src 'self';
      script-src 'self' 'unsafe-eval' 'unsafe-inline';
      style-src 'self' 'unsafe-inline';
      img-src * blob: data:;
      font-src 'self';
      connect-src *;
    `.replace(/
/g, ''),
  },
]
```

Validation des entrees — Schema Zod

lib/validations.ts

```
// lib/validations.ts
import { z } from 'zod'

export const loginSchema = z.object({
  email: z.string().email('Email invalide'),
  password: z.string().min(8, 'Minimum 8 caracteres')
})

export const createPostSchema = z.object({
  content: z.string().max(2000).optional(),
  mediaUrl: z.string().url().optional(),
  groupId: z.string().cuid().optional()
}).refine(data => data.content || data.mediaUrl, {
  message: 'Un post doit avoir du contenu ou un media'
})

export const reactionSchema = z.object({
  type: z.enum(['LIKE', 'LOVE', 'HAHA', 'WOW', 'SAD', 'ANGRY', 'CARE'])
})

export const createMessageSchema = z.object({
  content: z.string().min(1).max(5000),
  conversationId: z.string().cuid()
})
```

9. Déploiement

9.1 Infrastructure Docker — Développement local

Pour garantir la reproductibilité de l'environnement de développement, j'ai mis en place un fichier docker-compose.yml qui orchestre les services nécessaires localement :

docker-compose.yml

```
# docker-compose.yml
version: '3.8'

services:
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: socialnet
      POSTGRES_PASSWORD: ${DB_PASSWORD}
      POSTGRES_DB: socialnetwork
    ports:
      - '5432:5432'
    volumes:
      - postgres_data:/var/lib/postgresql/data

  redis:
    image: redis:7-alpine
    ports:
      - '6379:6379'
    command: redis-server --requirepass ${REDIS_PASSWORD}

  app:
    build: .
    ports:
      - '3000:3000'
    environment:
      DATABASE_URL: postgresql://socialnet:${DB_PASSWORD}@postgres:5432/socialnetwork
      REDIS_URL: redis://:${REDIS_PASSWORD}@redis:6379
    depends_on:
      - postgres
      - redis

volumes:
  postgres_data:
```

9.2 CI/CD et mise en production

Le pipeline de déploiement est entièrement automatisé via GitHub Actions. Chaque push sur la branche main déclenche la chaîne suivante :

- Étape 1 — Lint TypeScript (tsc --noEmit) — vérifie l'absence d'erreurs de typage
- Étape 2 — Tests d'intégration (vitest) — exécutés contre une BDD PostgreSQL éphémère
- Étape 3 — Build Next.js (next build) — valide que le bundle est compilable
- Étape 4 — Deploy Vercel automatique — si tous les checks passent
- Étape 5 — Migration Prisma en production (prisma migrate deploy) — sans downtime

deploy.yml

```
# .github/workflows/deploy.yml
name: CI/CD
on:
  push:
    branches: [main]
jobs:
  test-and-deploy:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16
        env:
          POSTGRES_PASSWORD: test
        ports: ['5432:5432']
    steps:
      - uses: actions/checkout@v4
      - uses: oven-sh/setup-bun@v1
        with: { bun-version: 'latest' }
      - run: bun install --frozen-lockfile
      - run: bunx prisma migrate deploy
      - run: bun run test
      - uses: amondnet/vercel-action@v25
        with:
          vercel-token: ${ secrets.VERCEL_TOKEN }
          vercel-args: '-prod'
```

Variables d'environnement de production

Variable	Rôle	Source
DATABASE_URL	Connexion PostgreSQL Neon	Neon Dashboard
UPSTASH_REDIS_URL	Redis temps réel Upstash	Upstash Console
UPSTASH_REDIS_TOKEN	Token auth Redis	Upstash Console
JWT_SECRET	Clé de signature JWT (256 bits)	Générée via openssl rand
CLOUDINARY_URL	URL + clé API Cloudinary	Cloudinary Dashboard
GOOGLE_CLIENT_ID	OAuth Google Client ID	Google Cloud Console
GOOGLE_CLIENT_SECRET	OAuth Google Secret	Google Cloud Console
NEXTAUTH_SECRET	Secret sessions Next.js	Générée via openssl rand

10. Recherche technique personnelle

10.1 Approche de veille

Le projet m'a conduit à explorer des domaines techniques souvent peu abordés en formation classique. Ma démarche de veille est structurée autour de quatre axes : lire la documentation officielle, tester en pratique, comparer les approches alternatives, et m'appuyer sur des retours d'expérience de la communauté.

Thématique	Problème rencontré	Solution explorée
SSE vs WebSockets	Vercel ne supporte pas les WebSockets durables en serverless	SSE + Redis pub/sub : compatible edge, moins de latence que polling
React Server Components	Comprendre la frontière client/serveur dans App Router	Lecture RFC RSC, expérimentation avec Suspense et streaming SSR
Pagination curseur	Pagination par offset inefficace sur grandes tables	Cursor-based pagination avec Prisma : stable, performant sur index
Prisma N+1	Requêtes multiples pour les relations imbriquées	include vs select Prisma, dataloader pattern, requêtes agrégées
JWT vs Sessions DB	Choisir entre stateless JWT et sessions stockées	JWT httpOnly pour simplicité serverless, refresh via reconnexion 7j
Cloudinary upload	Upload sécurisé sans exposer les clés API côté client	Signed upload preset : le serveur génère une signature temporaire

10.2 Analyse comparative — Technologies temps réel

Le choix de la solution temps réel est l'une des décisions architecturales les plus importantes du projet. J'ai comparé trois approches :

Critère	WebSockets	SSE + Redis	Long Polling
Compatibilité Vercel/Edge	✗ Non	✓ Oui	✓ Oui
Bidirectionnel natif	✓ Oui	✗ Non (push only)	✗ Non
Latence	< 10ms	< 100ms	1-3s
Complexité implémentation	Élevée (serveur dédié)	Moyenne	Faible
Scalabilité serverless	Difficile	✓ Excellente	Correcte
Choix retenu	—	✓ CHOISI	Fallback

10.3 Sources mobilisées

- Documentation officielle : Next.js docs, Prisma docs, Upstash Redis, Vercel documentation
- Articles techniques : Lee Robinson (Vercel), Josh W. Comeau, Kent C. Dodds (testing)
- Conférences et talks : React Conf 2024 (RSC deep dive), Next.js Conf
- GitHub : Issues Prisma, discussions Next.js App Router RFC, exemples officiels
- Communauté : Discord Next.js, r/nextjs, Stack Overflow pour les cas limites
- Expérimentation directe : Branches de test, prototypes isolés avant intégration

11. Bilan RNCP et conclusion

11.1 Couverture des compétences RNCP 37873

Le projet Social Network couvre l'intégralité des compétences attendues du titre professionnel CDA (RNCP 37873). Voici la correspondance détaillée :

Compétence RNCP	Réalisation concrète	Niveau
Développer des composants d'accès aux données	18 modèles Prisma, 60+ requêtes optimisées, migrations, seed	✓ Validé
Concevoir une IHM adaptée au contexte	80+ composants React, responsive mobile-first, design system Tailwind	✓ Validé
Développer la partie back-end d'une interface web	60+ endpoints API REST, Next.js API Routes, validation Zod, gestion erreurs	✓ Validé
Mettre en place une base de données	PostgreSQL via Prisma, 18 entités, 40+ relations, contraintes, index	✓ Validé
Développer des composants dans le langage de l'environnement	TypeScript strict, composants Server et Client, hooks React, utilitaires	✓ Validé
Préparer et exécuter les plans de tests	35+ jeux d'essai documentés, tests intégration Supertest, Playwright E2E	✓ Validé
Préparer et documenter le déploiement	Docker Compose, variables env, CI/CD GitHub Actions, checklist production	✓ Validé
Contribuer à la mise en production	Déploiement Vercel, migration Prisma prod, monitoring, rollback plan	✓ Validé
Concevoir une application en couches	App Router Next.js, séparation UI/API/Data, middleware, services	✓ Validé
Utiliser des composants d'accès aux données	Prisma client type-safe, Redis Upstash SDK, Cloudinary SDK	✓ Validé
Développer une partie front-end sécurisée	JWT httpOnly, CSRF protection, validation côté client et serveur, HTTPS	✓ Validé

11.2 Bilan personnel

Ce projet représente pour moi bien plus qu'un exercice de certification. C'est la preuve que ma reconversion est aboutie et que je suis capable de livrer une application complexe, sécurisée, et documentée, de A à Z.

La complexité du projet — 18 modèles de données, des flux temps réel, une API de 60+ endpoints, des contraintes d'hébergement serverless — m'a forcé à aller chercher des solutions que la formation seule n'enseigne pas. J'ai appris à résoudre des problèmes réels : optimiser des requêtes qui créaient des timeouts, implémenter du temps réel sans WebSocket, concevoir un système de notification multi-types sans accumuler la dette technique.

Ma reconversion depuis l'industrie m'a apporté une perspective précieuse : dans les métiers techniques, la fiabilité compte plus que la sophistication. Un code maintenable, testé, documenté et déployable vaut infiniment plus qu'une feature brillante fragile. C'est avec cette conviction que j'aborde ma recherche d'alternance Bac+5.

11.3 Forces et axes d'amélioration

✓ Points forts	🔴 Axes d'amélioration
• Architecture API robuste et scalable	• Tests unitaires encore insuffisants
• Maîtrise de Prisma et PostgreSQL	• Rate limiting non implémenté (identifié)
• Système temps réel SSE + Redis fonctionnel	• WebSockets à explorer en environnement dédié
• Couverture de test documentée	• Internationalisation (i18n) absente
• TypeScript strict bout en bout	• Tests de charge non réalisés
• Déploiement Docker + CI/CD opérationnel	• Accessibilité (WCAG) partiellement couverte

11.4 Vision et perspectives

Ce projet marque la fin d'une première étape et le début d'une trajectoire que j'ai décidée de poursuivre en alternance Bac+5 Chef de projet, Architecture et Green IT. Je souhaite approfondir mes compétences en architecture de systèmes distribués, en optimisation de performances à l'échelle, et en pratiques DevOps avancées, avec une attention particulière aux enjeux de sobriété numérique.

Les domaines qui m'attirent particulièrement pour la suite sont les architectures event-driven (Kafka, RabbitMQ), l'observabilité (OpenTelemetry, Grafana), la sécurité applicative approfondie ainsi que les approches Green IT visant à concevoir des systèmes plus durables et efficaces. Ma reconversion depuis l'industrie m'a appris que la fiabilité n'est pas un luxe — c'est le standard minimum.

"Ce dossier est le reflet d'une conviction : on n'apprend pas à développer en lisant, on apprend en construisant quelque chose qui résiste à la réalité."

Christophe Lecart — Zone01 Rouen Normandie — Session CDA 02 Juin 2026